

Федеральное государственное автономное образовательное учреждение высшего
образования «Санкт-Петербургский политехнический университет Петра
Великого»

Отчет по практическому занятию
«Наука о данных и аналитика больших объемов информации»

Практическая работа 9
«ВИЗУАЛИЗАЦИЯ ДАННЫХ В GRAFANA
ЧЕРЕЗ PROMETHEUS»
«ВИЗУАЛИЗАЦИЯ ДАННЫХ В GRAFANA
БЕЗ ИСПОЛЬЗОВАНИЯ PROMETHEUS»

Выполнил работу
ст. гр. 3740105/50101
Белаш Андрей Анатольевич
Проверил преподаватель
Соломонов Максим Сергеевич

Санкт-Петербург,
2026

СОДЕРЖАНИЕ

1. ХОД РАБОТЫ	4
1.1. Установка и настройка NGINX	4
1.2 Установка nginx-prometheus-exporter.....	5
1.3. Настройка Prometheus	6
1.4. Настройка Grafana	7
1.5 Визуализация данных из PostgreSQL. Установка и настройка PostgreSQL	9
1.6 Настройка подключения в Grafana	10
2. РЕЗУЛЬТАТЫ РАБОТЫ	13
2.1. Проверка работоспособности	13
2.2. Созданные артефакты	13
ВЫВОДЫ	14

ВВЕДЕНИЕ

Визуализация данных — критически важный этап анализа и мониторинга систем. Grafana предоставляет гибкие возможности для создания интерактивных дашбордов из различных источников данных.

Существует два основных подхода к сбору метрик для Grafana:

1. Через Prometheus:

- используется система мониторинга Prometheus как промежуточное звено;
- экспортеры (например, `nginx-prometheus-exporter`) собирают метрики с приложений;

- Prometheus периодически опрашивает экспортеры и хранит метрики;
- Grafana запрашивает данные из Prometheus.

2. Напрямую из базы данных:

- Grafana подключается напрямую к базе данных (PostgreSQL);
- метрики загружаются непосредственно в БД;
- Grafana выполняет SQL-запросы для получения данных;
- Prometheus не используется.

В данных заданиях изучаются оба подхода на примерах мониторинга NGINX-сервера и визуализации данных из PostgreSQL.

Цель: изучить два подхода к визуализации данных в Grafana — через Prometheus и напрямую из базы данных PostgreSQL.

Задачи:

- настроить сбор метрик NGINX через `nginx-prometheus-exporter`;
- интегрировать NGINX exporter с Prometheus;
- создать визуализации метрик NGINX в Grafana;
- установить и настроить PostgreSQL;
- настроить прямое подключение Grafana к PostgreSQL;
- создать таблицу с данными и визуализировать их в Grafana.

1. ХОД РАБОТЫ

1.1. Установка и настройка NGINX

Шаг 1. Установка NGINX. Открытие терминала WSL и выполнение: `sudo apt install nginx -y`.

Шаг 2. Настройка логирования. Откройте конфигурационный файл NGINX: `sudo nano /etc/nginx/nginx.conf`. В раздел `http` добавление custom формат логирования:

```
log_format custom '$remote_addr - $remote_user [$time_local] '
    '$request' $status $body_bytes_sent '
    '$http_referer' '$http_user_agent' '
    '$http_x_forwarded_for';
```

```
access_log /var/log/nginx/access.log custom;
```

Шаг 3. Перезапуск NGINX. Применения изменения: `sudo nginx -s reload`. Проверка работоспособности: `sudo systemctl status nginx` (рис. 1).

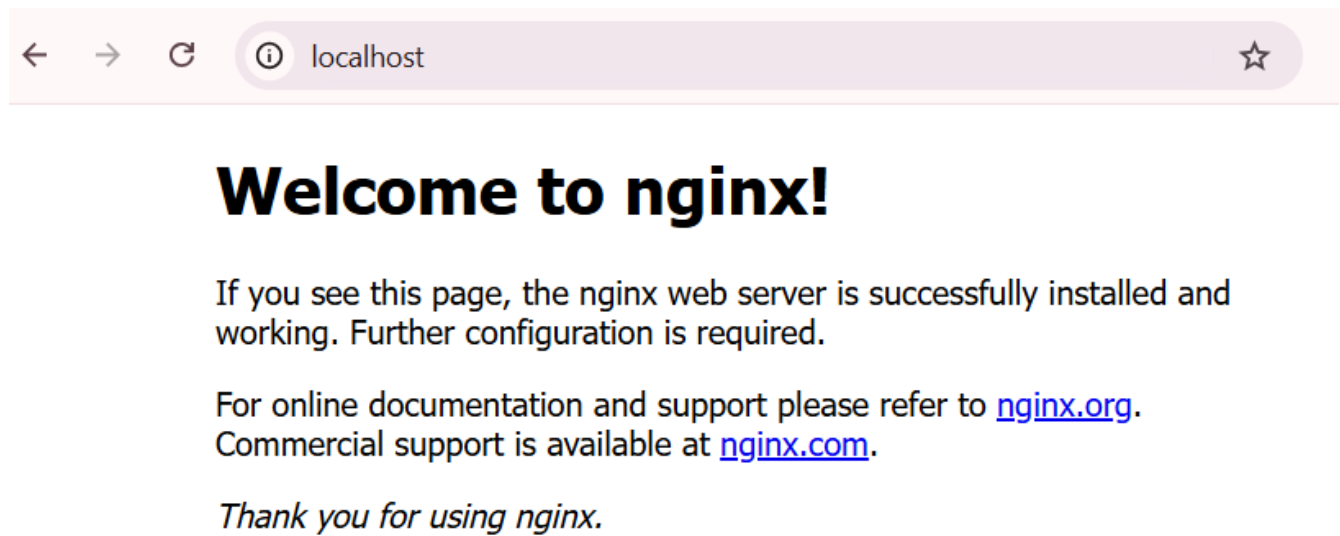
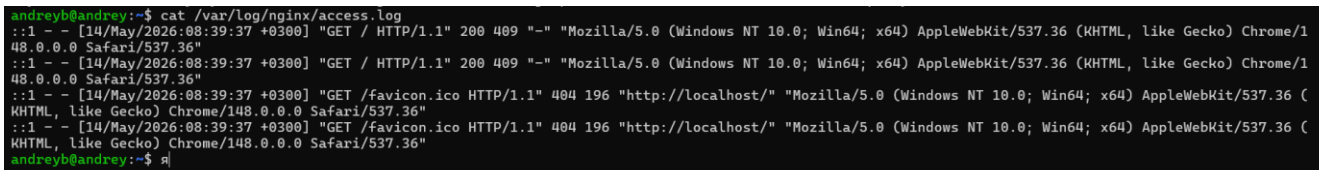


Рисунок 1 – Страница NGINX

Шаг 4. Проверка логов. Посещение страницы NGINX несколько раз, затем проверка логов: `cat /var/log/nginx/access.log` (рис. 2)



```
andreyb@andrey:~$ cat /var/log/nginx/access.log
::1 - - [14/May/2026:08:39:37 +0300] "GET / HTTP/1.1" 200 409 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
::1 - - [14/May/2026:08:39:37 +0300] "GET / HTTP/1.1" 200 409 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
::1 - - [14/May/2026:08:39:37 +0300] "GET /favicon.ico HTTP/1.1" 404 196 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
::1 - - [14/May/2026:08:39:37 +0300] "GET /favicon.ico HTTP/1.1" 404 196 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
andreyb@andrey:~$
```

Рисунок 2 – Лог-файл NGINX

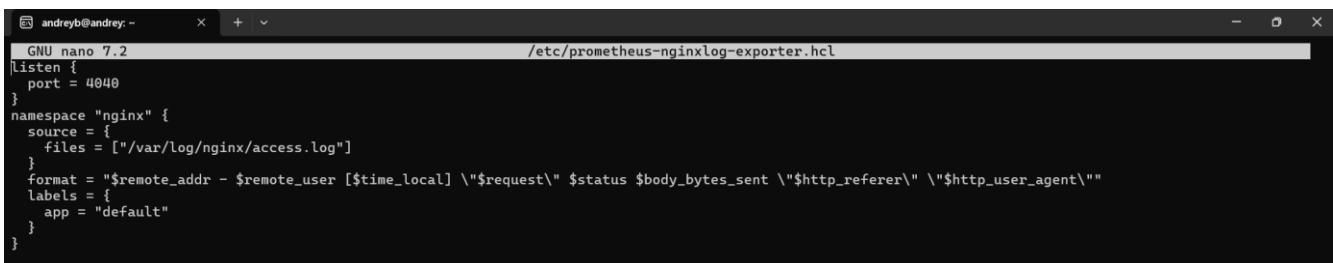
1.2 Установка `nginx-prometheus-exporter`: `cd /mnt/c/Users/andreyb/Downloads`

Шаг 5. Скачивание экспортера: `wget https://github.com/martin-helmich/prometheus-nginxlog-exporter/releases/download/v1.8.0/prometheus-nginxlog-exporter\_1.8.0\_linux\_amd64.deb`

Установка пакета: `sudo apt install ./prometheus-nginxlog-exporter_1.8.0_linux_amd64.deb`

Шаг 6. Настройка экспортера. Открытие конфигурационного файла: `sudo nano /etc/prometheus-nginxlog-exporter.hcl`

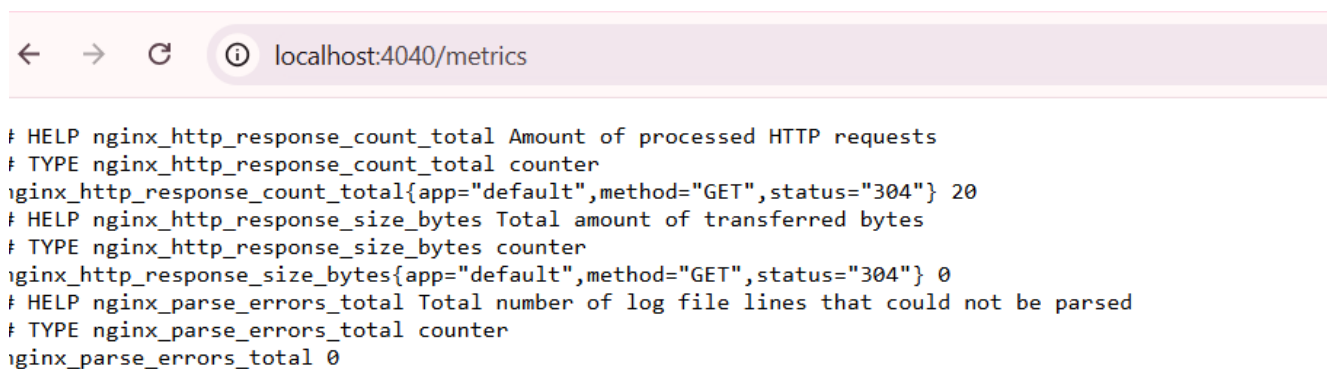
Изменение формата логирования: `format "$remote_addr - $remote_user [$time_local] \"$request\" $status $body_bytes_sent \"$http_referer\" \"$http_user_agent\" \"$http_x_forwarded_for\""` (рис. 3).



```
GNU nano 7.2 /etc/prometheus-nginxlog-exporter.hcl
listen {
  port = 4040
}
namespace "nginx" {
  source = {
    files = ["/var/log/nginx/access.log"]
  }
  format = "$remote_addr - $remote_user [$time_local] \"$request\" $status $body_bytes_sent \"$http_referer\" \"$http_user_agent\" \"$http_x_forwarded_for\""
  labels = {
    app = "default"
  }
}
```

Рисунок 3 – Конфигурация экспортера

Шаг 7. Запуск экспортера. `prometheus-nginxlog-exporter -config-file /etc/prometheus-nginxlog-exporter.hcl &`. Проверка метрик <http://localhost:4040/metrics> (рис. 4).



```
# HELP nginx_http_response_count_total Amount of processed HTTP requests
# TYPE nginx_http_response_count_total counter
nginx_http_response_count_total{app="default",method="GET",status="304"} 20
# HELP nginx_http_response_size_bytes Total amount of transferred bytes
# TYPE nginx_http_response_size_bytes counter
nginx_http_response_size_bytes{app="default",method="GET",status="304"} 0
# HELP nginx_parse_errors_total Total number of log file lines that could not be parsed
# TYPE nginx_parse_errors_total counter
nginx_parse_errors_total 0
```

Рисунок 4 – Метрики экспортера

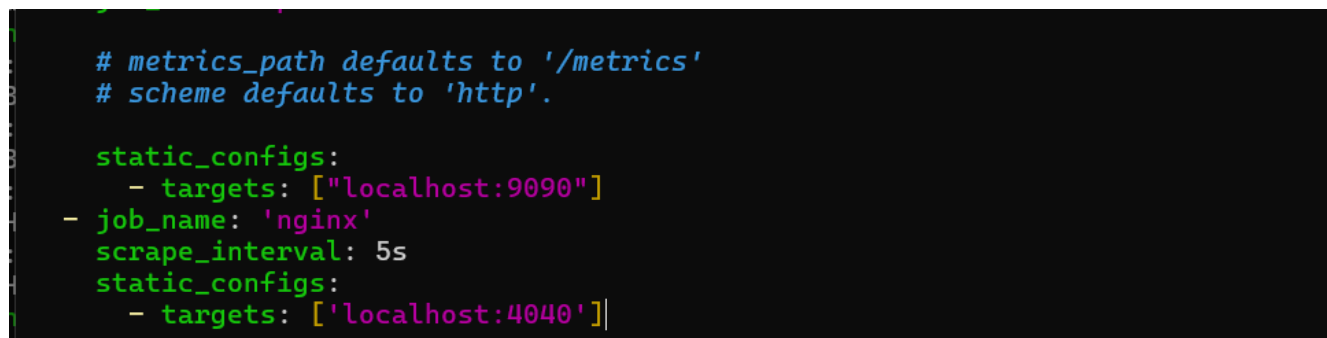
1.3. Настройка Prometheus

Шаг 8. Добавление экспортера в Prometheus

Переход в директорию Prometheus: `cd`

`/mnt/c/Users/andreyb/Downloads/prometheus-2.33.4.linux-amd64`

Добавьте новый job в раздел `scrape_configs`: (рис. 5).



```
# metrics_path defaults to '/metrics'
# scheme defaults to 'http'.

static_configs:
- targets: ["localhost:9090"]
- job_name: 'nginx'
  scrape_interval: 5s
  static_configs:
  - targets: ['localhost:4040']
```

Рисунок 5 – Конфигурация Prometheus

Шаг 9. Перезапуск Prometheus

`./Prometheus`. Проверка подключения в браузере: <http://localhost:9090/targets> (рис. 6).

nginx (1/1 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:4040/metrics	UP	<code>instance="localhost:4040"</code> <code>job="nginx"</code>	2.317s ago	1.894ms	

prometheus (1/1 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	<code>instance="localhost:9090"</code> <code>job="prometheus"</code>	12.956s ago	5.323ms	

Рисунок 6 – Targets в Prometheus

Шаг 10. Проверка запросов. Открытие главной страницы Prometheus: <http://localhost:9090>. Ввод запроса: `nginx_http_response_count_total{status="304"}`.

1.4. Настройка Grafana

Шаг 11. Создание панели в Grafana. Открытие Grafana: <http://localhost:3000>

1. Нажмите Create (+) → Dashboard
2. Нажмите Add new panel
3. В поле Query введите: `up{}`
4. Нажмите Execute
5. В поле Panel title введите: "NGINX Status"
6. Нажмите Apply (рис. 7).

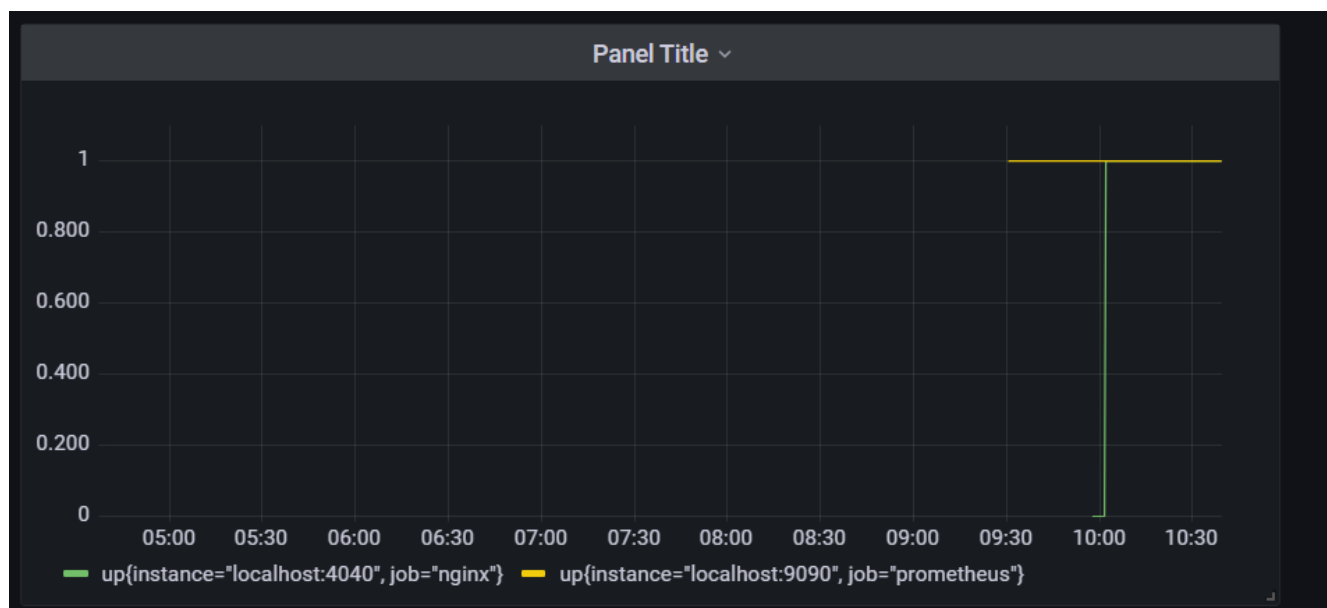


Рисунок 7 – Панель up {}

Шаг 12. Добавление панели с метриками NGINX

1. Нажмите Add → Visualization
2. В поле Query введите: `nginx_http_response_count_total{status="304"}`
3. Выберите визуализацию Time series
4. Настройте панель:
 - Title: «NGINX 304 Responses»
 - Description: «HTTP 304 status count»
5. Нажмите Apply
6. Нажмите Save dashboard
7. Введите название: «NGINX Metrics»
8. Нажмите Save

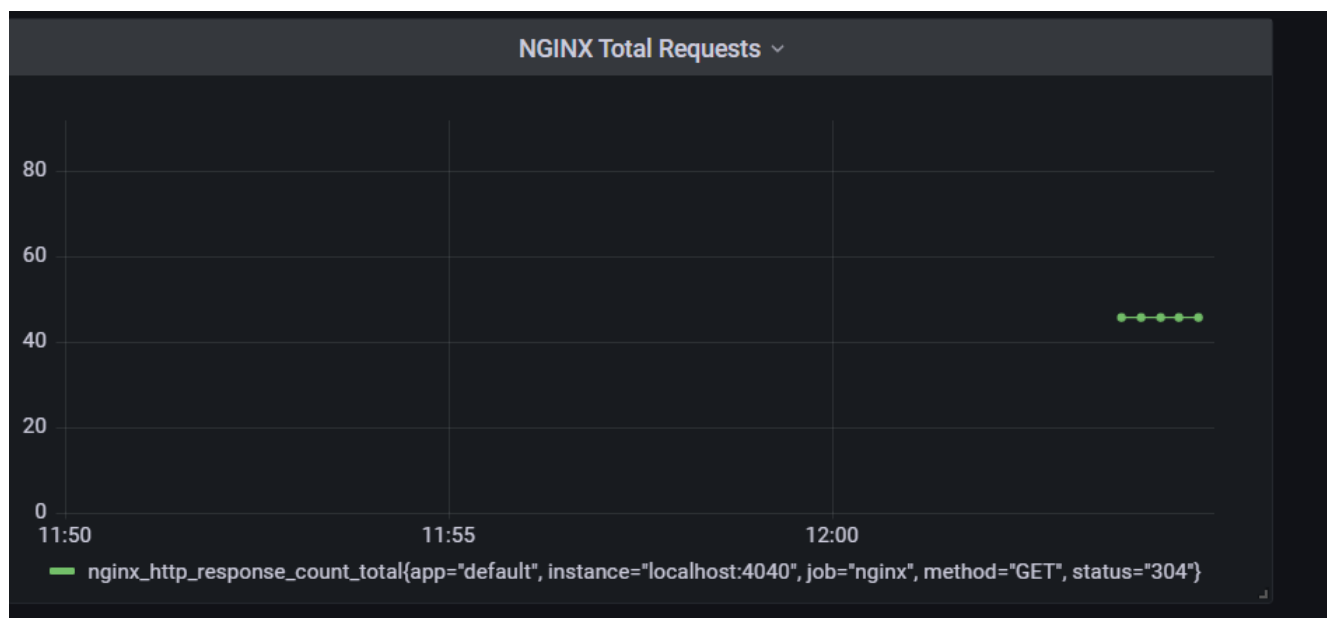


Рисунок 8 – Готовый дашборд с метриками NGINX

1.5 Визуализация данных из PostgreSQL. Установка и настройка PostgreSQL

Запуск и проверка PostgreSQL (рис. 9).

```
andreyb@andrey:~$ sudo service postgresql start
[sudo] password for andreyb:
andreyb@andrey:~$ sudo service postgresql status
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Thu 2026-05-14 08:38:58 MSK; 3h 29min ago
     Main PID: 1436 (code=exited, status=0/SUCCESS)
        CPU: 2ms

May 14 08:38:58 andrey systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 14 08:38:58 andrey systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
andreyb@andrey:~$ |
```

Рисунок 9 – PostgreSQL

Далее создаем таблицу (рис. 10)

```

ERROR: relation "playground" already exists
postgres=# SELECT * FROM playground;
 id |          ts          | val
----+-----
 1  | 2022-04-24 20:50:00 | 4
 2  | 2024-04-09 20:50:00 | 4
 3  | 2024-04-24 21:00:00 | 2
 4  | 2024-04-24 21:05:00 | 10
 5  | 2024-04-24 21:10:00 | 5
 6  | 2024-04-24 21:15:00 | 7
 7  | 2024-04-24 21:20:00 | 3
(7 rows)

postgres=# |

```

Рисунок 10 - Данные PostgreSQL

1.6 Настройка подключения в Grafana

Шаг 17. Добавление источника данных

Откройте Grafana: <http://localhost:3000>

В левом меню нажмите Configuration (шестеренка)

1. Выберите Data sources

2. Нажмите Add data source

3. Выберите PostgreSQL

4. Заполните параметры:

- Host: localhost:5432

- Database: postgres

- User: postgres

- Password: postgres

- TLS/SSL Mode: disable

- Нажмите Save & Test

- Должно появиться: «Data source is working». (рис. 11)

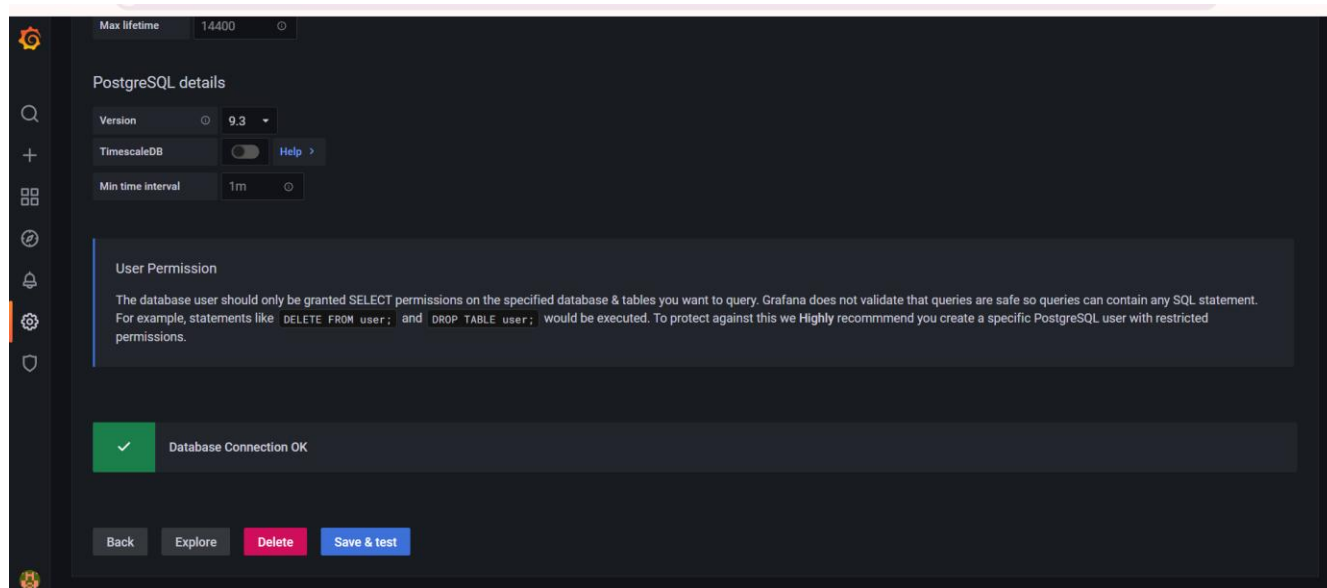


Рисунок 11 – Настройка PostgreSQL в Grafana

Шаг 18. Создание дашборда

Нажмите Create (+) → Dashboard

Нажмите Add new panel

В поле Data source выберите PostgreSQL

В поле Query введите SQL:

```
SELECT
```

```
  ts as time,
```

```
  val
```

```
FROM playground
```

```
WHERE $__timeFilter(ts)
```

```
ORDER BY ts
```

Просмотр графика <http://localhost:3000/dashboards> (рис. 12)

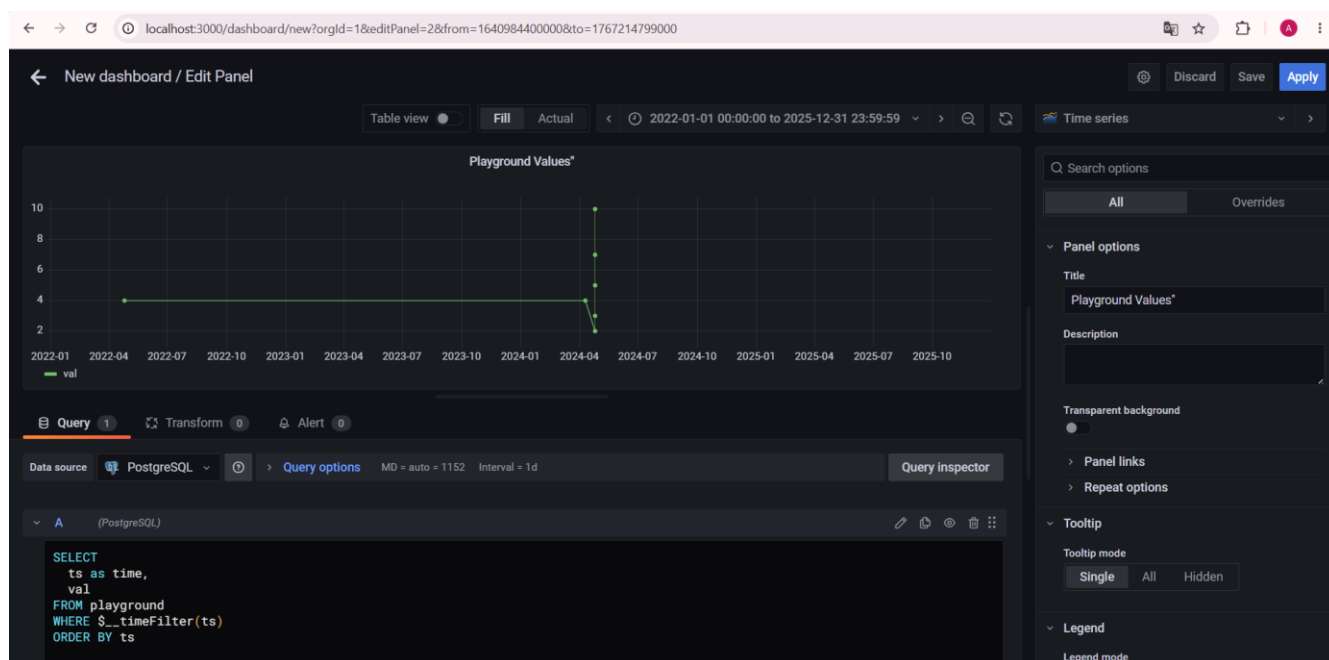


Рисунок 12 – График в Grafana

2. РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Проверка работоспособности

Задание 8 (NGINX + Prometheus):

- NGINX установлен и настроен;
- nginx-prometheus-exporter запущен на порту 4040;
- Prometheus собирает метрики с экспортера;
- В Grafana создан дашборд с метриками NGINX.

Задание 9 (PostgreSQL):

- PostgreSQL установлен и запущен;
- Создана таблица playground с данными;
- Grafana подключена напрямую к PostgreSQL;
- Создан дашборд с визуализацией данных из БД.

2.2. Созданные артефакты

Задание 8:

- Конфигурация NGINX с custom log_format;
- Dashboard "NGINX Metrics" в Grafana: панель up{} (статус доступности), панель nginx_http_response_count_total (счетчик ответов).

Задание 9: таблица playground в PostgreSQL, Dashboard «PostgreSQL Data» в Grafana: Панель с графиком значений из БД.

ВЫВОДЫ

В ходе выполнения практических заданий №8 и №9 были получены следующие навыки:

1) Работа с NGINX:

- установка и базовая настройка веб-сервера NGINX;
- настройка custom формата логирования;
- перезапуск службы NGINX.

2) Работа с экспортерами:

- установка nginx-prometheus-exporter;
- настройка экспортера для чтения логов NGINX;
- преобразование логов в метрики Prometheus.

3) Интеграция с Prometheus:

- добавление новых targets в конфигурацию Prometheus;
- проверка статуса сбора метрик;
- выполнение PromQL запросов к метрикам NGINX;

4) Работа с PostgreSQL:

- установка и запуск СУБД PostgreSQL;
- создание таблиц и вставка данных;
- выполнение SQL-запросов.

5) Визуализация в Grafana:

- создание панелей с разными источниками данных;
- настройка подключения к PostgreSQL;
- написание SQL-запросов для визуализации.

Выбор типов визуализации (Time series)

4.2. Сравнение подходов

Подход через Prometheus (Задание 8):

Преимущества:

- автоматический сбор метрик;
- хранение истории;
- мощный язык запросов PromQL;
- возможность настройки алертов.

Недостатки:

- требует дополнительных компонентов (экспортеры);
- сложнее в настройке.

Прямое подключение к БД:

Преимущества:

- проще в настройке;
- прямой доступ к данным;
- использование стандартного SQL.

Недостатки:

- нет автоматического сбора метрик;
- данные нужно загружать вручную;
- меньше возможностей для мониторинга в реальном времени.

4.3. Практическая значимость:

Изученные подходы позволяют:

- мониторить веб-серверы в реальном времени через Prometheus;
- визуализировать бизнес-данные напрямую из баз данных;
- выбирать оптимальный подход в зависимости от задачи;
- создавать комплексные дашборды с разными источниками данных.

4.4. Перспективы развития:

Полученные знания открывают возможности для:

- создания сложных дашбордов с множеством панелей;
- интеграции с другими источниками данных;
- автоматизации сбора и анализа метрик.